

Requisitos

Atendimento dos requisitos — Consulta de débitos veiculares

Requisitos funcionais e arquitetura

Suportar múltiplos provedores — Atendido

Há dois adapters de provedores:

- `app/Infrastructure/Providers/ProviderA/ProviderAAdapter.php` consulta o Provedor A, que retorna JSON.
- `app/Infrastructure/Providers/ProviderB/ProviderBAdapter.php` consulta o Provedor B, que retorna XML.

Ambos implementam `app/Infrastructure/Providers/Contracts/DebtProvider.php`. As URLs são configuradas por `PROVIDER_A_URL` e `PROVIDER_B_URL`.

Fazer fallback caso um provedor falhe — Atendido

`app/Application/Vehicle/GetVehicleDebts.php` percorre os provedores registrados na ordem definida em `app/Providers/AppServiceProvider.php`. Quando um provedor falha ou retorna dados inválidos, o caso de uso tenta o próximo.

Se todos falharem, o controller retorna HTTP 503.

Isolar integração, domínio e pagamento — Atendido

A solução separa essas responsabilidades:

- **Integração:** adapters em `app/Infrastructure/Providers/`.
- **Domínio:** débitos, tipos, dinheiro e regras de juros em `app/Domain/Debt/` e `app/Domain/Shared/`.
- **Pagamento:** estratégias PIX e cartão em `app/Domain/Payment/`.
- **Orquestração:** `app/Application/Vehicle/GetVehicleDebts.php`.
- **HTTP:** controller e middleware em `app/Http/`.

Facilitar a inclusão de provedores e regras de juros — Atendido com registro explícito

Um novo provedor implementa `DebtProvider` e é registrado em `AppServiceProvider.php`. O orquestrador continua consumindo a interface comum, sem precisar conhecer os detalhes do novo formato externo.

Um novo tipo de débito exige:

1. Adicionar o tipo ao enum `app/Domain/Debt/DebtType.php`.
2. Criar uma regra que implemente `app/Domain/Debt/Interest/InterestRule.php`.
3. Associar a regra ao tipo em `app/Domain/Debt/DebtCalculator.php`.

O registro de providers é explícito; não há descoberta automática de adapters.

Tratamento de erros

Atenção: os status HTTP previstos estão implementados, mas os corpos JSON atuais não correspondem exatamente aos exemplos planos do requisito.

Tipo de débito desconhecido — Parcial

O sistema retorna HTTP 422 quando encontra um tipo desconhecido. Porém, hoje interrompe a normalização ao encontrar **qualquer** débito desconhecido; não espera para verificar se **todos** os débitos retornados são desconhecidos.

O corpo atual tem este formato:

```
{
  "error": {
    "code": "unknown_debt_type",
    "message": "Unknown debt type: LICENSING"
  }
}
```

O requisito especifica:

```
{"error": "unknown_debt_type"}
```

A implementação está em `app/Application/Vehicle/GetVehicleDebts.php` e `app/Http/Controllers/VehicleDebtController.php`.

Todos os provedores indisponíveis — Parcial

Quando todos os provedores falham, o sistema retorna HTTP 503. O código atual da resposta é `providers_unavailable`, dentro de um objeto:

```
{
  "error": {
    "code": "providers_unavailable",
    "message": "Debt providers are temporarily unavailable."
  }
}
```

O formato solicitado é:

```
{"error": "all_providers_unavailable"}
```

A conversão para HTTP 503 está no `VehicleDebtController.php`.

Placa inválida — Parcial

`app/Domain/Vehicle/Plate.php` valida placas nos formatos antigo (`ABC1234`) e Mercosul (`ABC1D23`). Placas inválidas recebem HTTP 400.

O corpo atual é:

```
{
  "error": {
    "code": "invalid_plate",
    "message": "..."
  }
}
```

O formato solicitado é:

```
{"error": "invalid_plate"}
```

A validação e resposta HTTP estão em

`app/Http/Controllers/VehicleDebtController.php`.

Casos de borda

Zero débitos — Atendido

Uma lista JSON vazia ou um XML `<debts/>` é considerada uma resposta válida. O serviço retorna `debts: []` e total `"0.00"`.

O mock local também oferece esse cenário com a placa `ZER0000`.

Todos os provedores falham — Atendido

O Postman usa `ZZZ9999` para simular indisponibilidade nos dois mocks e verificar HTTP 503.

Tipo desconhecido — Parcial

Há cobertura para receber um tipo não reconhecido e retornar 422. Ainda falta cobrir a condição específica do requisito: **todos** os débitos desconhecidos, inclusive o comportamento quando a resposta mistura tipos conhecidos e desconhecidos.

Débito não vencido — Atendido

As regras de juros limitam os dias de atraso a zero. Assim, uma dívida com vencimento futuro ou no dia da consulta não recebe juros.

Placa fora do padrão — Atendido

A placa é validada antes das chamadas aos provedores. Uma placa fora dos formatos antigos e Mercosul retorna HTTP 400.

Dados divergentes entre provedores — Estratégia documentada, sem reconciliação

O serviço usa o primeiro provedor que retorna uma resposta válida. Não compara os valores dos provedores nem tenta reconciliá-los, pois não há regra definida para resolver divergências. A estratégia está descrita no `README.md`.

Requisitos desejáveis

Simulação de falhas — Atendido

O mock em `tools/mock_providers.py` permite exercitar:

- `ZZZ0000`: Provedor A falha e Provedor B é consultado.
- `ZZZ9999`: ambos falham.

- `ZER0000` : resposta sem débitos.

Timeout, retry e circuit breaker — Parcial

Os adapters têm timeout configurável — três segundos por padrão — e retries limitados com intervalo de 150 ms. O intervalo não é exponencial e não há circuit breaker implementado.

Testes automatizados — Implementados; execução não confirmada

Os testes PHP estão em `tests/Unit/` e `tests/Feature/`. Eles cobrem regras de juros, placa, consulta JSON, fallback XML, zero débitos e indisponibilidade dos provedores.

A coleção `postman/vehicle-debt-api.postman_collection.json` também cobre os fluxos HTTP. A existência dos testes não significa que tenham sido executados neste ambiente.

Logs estruturados e mascaramento de placa — Atendido

Os logs registram eventos e contexto, como provider, quantidade de débitos e tipo de falha. A placa é mascarada nos dados de log (`ABC****`), evitando gravar o valor completo.

A implementação está em `app/Application/Vehicle/GetVehicleDebts.php` e `app/Http/Controllers/VehicleDebtController.php`.

Tamanho de requisição e campos desconhecidos — Atendido

O limite está configurado em 16 KiB — menor que o exemplo de 1 MiB — tanto no Nginx quanto no middleware `app/Http/Middleware/RejectOversizedJson.php`.

O controller rejeita campos JSON que não sejam `plate` ou `debt_type`.

Requests relevantes da coleção Postman

Importar `postman/vehicle-debt-api.postman_collection.json` e o ambiente `postman/vehicle-debt-local.postman_environment.json`.

- **Provider fallback: A fails, B returns XML:** fallback do Provedor A para B.
- **All providers unavailable (503):** indisponibilidade dos dois provedores.
- **Search debts: zero-debt vehicle:** retorno válido sem débitos.
- **Invalid plate (400):** rejeição de placa inválida.

- **Unknown debt type (422):** rejeição de filtro não reconhecido.
 - **Search debts: SOMENTE_IPVA / SOMENTE_MULTA:** seleção por tipo de débito.
-